

Predicciones JS Moderno

Camila Sepúlveda Venegas

22 de agosto 2025

MERN julio 2025

Bloque 1

```
const info = { //Declara constante "info"
  personal: { //Compuesta por un objeto "personal"
    nombre: 'Carlos', //Que contiene las keys "nombre", "apellido" y "detalles"
    apellido: 'Vega',
    detalles: { //Esta key es, a su vez, un objeto
      edad: 30, //Formado por las keys "edad" y "ocupación"
      ocupación: 'Ingeniero'
    }
  }
};

const { personal: { detalles: { edad, salario } } } = info;
/*Acá se declara una constante nueva, mediante deestructuración,
usando de referencia la constante "info". Dentro de "info",
se revisa "personal" y se obtiene la key "edad";
creándose la key "salario" sin datos asociados.*/
console.log(edad); //Imprimirá "30"
console.log(salario); //Imprimirá "undefined",
puesto que existe la key pero no tiene un valor asignado.*/
```

```
PS D:\Descargas\Coding Dojo\Bootcamp\skillne
30
undefined
PS D:\Descargas\Coding Dojo\Bootcamp\skillne
```

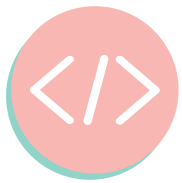
Después de ejecutar el código, no hubo diferencias entre la predicción y los resultados obtenidos.

Bloque 2

```
const objetoA = { a: 1, b: 2, c: 3 };
//Declara constante "objetoA", compuesto por las keys a, b y c.
const objetoB = { b: 4, c: 5, d: 6 };
//Declara constante "objetoB", compuesto por las keys b, c y d.
const resultado = { ...objetoA, ...objetoB };
//Declara constante "resultado", haciendo una copia de "objetoA" y "objetoB"
console.log(resultado);
//Imprimirá las keys a, b, c y d (con sus respectivos valores),
// aunque b y c se actualizarán de 2 y 3 a 4 y 5, respectivamente.
```

```
PS D:\Descargas\Coding Dojo\Bootcamp\ski
{ a: 1, b: 4, c: 5, d: 6 }
PS D:\Descargas\Coding Dojo\Bootcamp\ski
```

Después de ejecutar el código, no hubo diferencias entre la predicción y los resultados obtenidos.



Predicciones JS Moderno

Camila Sepúlveda Venegas

22 de agosto 2025

MERN julio 2025

Bloque 3

```
const verificar = () => { //Declara constante "verificar" y le entrega
  //una función flecha anónima
  if (true) { //Con la condicional de que si es "true",
    // entonces se ejecute el bloque de código a continuación
    const a = 2; //Crea una constante "a" de valor 2
    let b = 3; //Una let "b" de valor 3
    var c = 4; //Y una variable "c" de valor 4
  }
  console.log(c); //En principio, cada uno de estos console.log
  //debe imprimir los valores asignados en el if, considerando
  //que se encuentra dentro de un mismo bloque de código*/
  console.log(a);
  console.log(b);
}
verificar(); //Pero al llamar a la constante "verificar", al no entregarle
//parámetros, como se puede corroborar si es "true"? (...)
//Por lo demás, considerando que se quiere invocar una let y una const
//fuera de su bloque, lo esperable sería que sólo se haga el console.log de "c"
//que es una var y, por ende, puede ser elevada y retornar un valor.*//
```

```
4
D:\Descargas\Coding Dojo\Bootcamp\sk
console.log(a);
^
ReferenceError: a is not defined
```

Después de ejecutar el código, se aprecia que acerté accidentalmente en la predicción. Mi suposición era que no se podría ejecutar el código, sin embargo, si corrió... hasta que se topó con el console.log(a) y el error rompió lo demás.

Bloque 4

```
const datos = Object.freeze({ nombre: 'Luis', edad: 29 });
//Declara constante "datos", la que congela
//las keys "nombre" y "edad", juntos a sus respectivos valores.
datos.edad = 30;
//En esta línea, se intenta actualizar el valor de "edad" a 30, pero...
console.log(datos.edad);
//Al hacer este console.log, y como consecuencia del uso del método freeze,
//lo que se imprimirá en consola es el dato y valor originales (léase edad: "29").
```

```
PS D:\Descargas\Coding Dojo\Boo
29
```

Después de ejecutar el código, no hubo diferencias entre la predicción y el resultado obtenido.



Predicciones JS Moderno

Camila Sepúlveda Venegas

22 de agosto 2025

MERN julio 2025

Bloque 5

```
const original = [1, 2, 3]; // Se declara constante "original",
// formada por un array de 3 elementos (1, 2 y 3).
const nuevo = original.concat(4); // Se declara constante "nuevo", la que se define
// como la concatenación del array original más un nuevo elemento (4).
console.log(original); // Sin embargo, el método .concat no modifica
// los arrays iniciales (y se supone que se usa para concatenar arrays,
// generando uno nuevo con la combinación de los originales).
console.log(nuevo); // Por ende, el console.log de "original" devolverá [1,2,3]
// y "nuevo"... no tengo certeza de que sea [1,2,3,4], por el hecho de que
// se deberían usar arrays para implementar el método (y no un array y un dato suelto)
```

```
[ 1, 2, 3 ]
[ 1, 2, 3, 4 ]
```

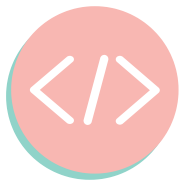
Después de ejecutar el código, se aprecia que el uso del método `.concat` no es estricto respecto de lo que se puede concatenar (como supuse inicialmente).

Bloque 6

```
const frutas = ['manzana', 'naranja', 'pera', 'mango'];
// Se declara constante "frutas", siendo un array de 4 elementos (manzana, naranja,
// pera, mango).
const [primera, segunda] = frutas;
// Se declara una nueva constante, haciendo una deestructuración de "frutas",
// capturando el primer y segundo elemento.
console.log(primera); // Imprimirá "manzana"
console.log(segunda); // Imprimirá "naranja"
```

```
manzana
naranja
```

Después de ejecutar el código, no hubo diferencias entre la predicción y el resultado obtenido.



Predicciones JS Moderno

Camila Sepúlveda Venegas

22 de agosto 2025

MERN julio 2025

Bloque 7

```
for (let i = 0; i < 3; i++) { //Se establece un bucle "for" que parte
  // evaluando en 0, lo hace hasta 2 y en cada vuelta suma 1 a la let "i".
  for (let i = 0; i < 2; i++) { //Se establece un nuevo bucle for (anidado),
    // el que parte en 0, llega hasta 1 y en cada vuelta suma 1 a la let "i".
    console.log(i); //Se imprimirá la let "i", con el valor asignado
    // en cada vuelta-repetición (en este caso, 0 y 1 en el "for" interno
    // y 0, 1 y 2 en el "for" externo).
    //Sim embargo, no está demás mencionar que no es una buena práctica usar

    //ambas let con el mismo nombre ("i"), puesto que puede provocar algún error
    // y dificulta la lectura e interpretación.
  }
}
```

0
1
0
1
0
1

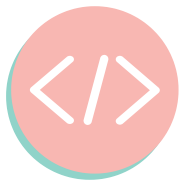
Después de ejecutar el código, noté un error en mi interpretación, dado que el "for" externo estableció cuántas veces se ejecuta el "for" interno. Originalmente, los analicé por separado y supuse que se imprimirían de ese modo.

Bloque 8

```
const numeros1 = [1, 2, 3]; //Se declara constante "numeros1", compuesta por
// un array de 3 elementos (1, 2 y 3)
const numeros2 = [3, 4, 5]; //Se declara constante "numeros2", también compuesta
// por un array de 3 elementos (3, 4 y 5)
const combinados = [...numeros1, ...numeros2];
//Se declara la constante "combinados", creando un nuevo array y haciendo una copia
// de las dos constantes anteriores.
console.log(combinados); //Creo que se imprimirá [1, 2, 3, 3, 4, 5] dado que,
// a diferencia de un objeto en donde se puede indicar mediante el uso de
// una misma key que se quiere sobrescribir un dato, en un array no hay
// como establecerlo de forma explícita (y si el compilador lo hiciera
// por cuenta propia, podría explotar alguna parte de un código?).
```

[1, 2, 3, 3, 4, 5]

Después de ejecutar el código, no hubo diferencias entre la predicción y el resultado obtenido.



Predicciones JS Moderno

Camila Sepúlveda Venegas

22 de agosto 2025

MERN julio 2025

Bloque 9

```
const demostracion = () => { //Se declara la constante "demostración", seguida por
  // una arrow function anónima
  var nombre = 'Ana'; //Se declara variable "nombre" de valor "Ana"
  let edad = 25; //Se declara let "edad" de valor "25"
  if (true) { //Se declara condicional "if", con el parámetro booleano "true"
    var nombre = 'Luis'; //Se declara variable "nombre" de valor "Luis"
    let edad = 30; //Se declara let "edad" de valor "30"
  }
  console.log(nombre); //Dado que "Luis" es una var, puede ser elevada fuera de
  // un bloque de código...
  console.log(edad);
  //No así "edad", que al ser una let conserva su valor y validez
  // dentro de su respectivo bloque de código.
  //Por ende, y considerando que los console.log se hacen fuera del if,
  // el primero imprimirá "Luis" y el segundo "25".
}
demostracion(); //Se invoca la constante "demostración", sin entregarle
// parámetros específicos.
```

Luis
25

Después de ejecutar el código, no hubo diferencias entre la predicción y el resultado obtenido.

Comentarios finales:

Aunque (estrictamente hablando) pude acertar 6/9 predicciones, agradezco la dinámica que plantearon para este ejercicio: tuve la oportunidad de comparar mis 'educated guesses' con los resultados efectivamente obtenidos y, mediante un nuevo análisis y/o lecturas adicionales, comprender en qué me equivoqué o que no interpreté bien. También pude darme cuenta que en ciertos puntos, sabía más de lo que pensaba... en fin, win/win por donde se le mire :)